Doctor

Doctor

`gemstone-js-doctor` is a local setup check for GemStone connection configuration and the optional native backend.

By default it does not connect to GemStone. It checks:

- the JavaScript runtime
- `GS_USERNAME`/`GS_USER` and `GS_PASSWORD`/`GS_PASS`
- `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE` library discovery
- whether `@gemstone-js/native` can be imported
- whether `@gemstone-js/native` has `createGciSessionWorker()` when `GS_NATIVE_SESSION_WORKER=1` is set
- whether the native `GciSessionWorker` prototype exposes the methods required by the `node-worker` backend, without starting a worker thread

```
gemstone-js-doctor
gemstone-js-doctor --json
gemstone-js-doctor --no-native
```

Use `--live` when you want it to log in and evaluate `1 + 1`:

```
gemstone-js-doctor --live
gemstone-js-doctor --live --json
```

The JSON output masks secrets. It reports whether credentials are set, but it does not print passwords or host passwords.

`Session.configFromEnv()` and `gemstone-js-doctor` prefer the canonical JavaScript names, but also accept the Pharo bridge live-test aliases:

Canonical	Compatibility alias
`GS_USERNAME`	`GS_USER`
`GS_PASSWORD`	`GS_PASS`
`GS_HOST`	`GS_NETLDI_HOST`
`GS_NETLDI`	`GS_NETLDI_NAME_OR_PORT`
`GS_GEM_SERVICE`	`GS_SERVICE`

If a canonical variable and its alias are both set to different non-empty values, doctor reports a warning and keeps the canonical value. The warning names the variables but does not print credential values. For password rotations, this means `GS_PASSWORD` must be changed or unset before `GS_PASS` can take effect.

Applications and setup tools can reuse the same policy with `sessionConfigFromEnv()` and `sessionEnvAliasConflicts()` from the public package API.

`GS_NATIVE_SESSION_WORKER=1` enables the optional Node worker backend used by `Session.connect({ nativeSessionWorker: true })`. That backend requires an `@gemstone-js/native` build that exports `createGciSessionWorker()` and a complete `GciSessionWorker` method surface. If setup fails with a missing worker export or missing worker methods, update the native package or clear `GS_NATIVE_SESSION_WORKER` to use the raw `Gci` backend while troubleshooting. Some GemStone client libraries omit optional GCI helpers such as transaction state probes; those should be treated as capability gaps rather than login configuration failures.

Beta Troubleshooting

- Run `gemstone-js-doctor --no-native` when validating docs, examples, or mock runtime behavior on a machine without GemStone client libraries.
- Run `gemstone-js-doctor --live --json` when login fails; the JSON output keeps secrets masked while showing which non-secret environment values were used.
- Run with `GS_NATIVE_SESSION_WORKER=1` before beta production trials to prove the installed native package exposes the worker backend expected by `gemstone-js`.
- If an installed package behaves differently from a checkout, use `npm run native-install:check` from the checkout. It packs both JS and native packages, installs them into a disposable project, and verifies the installed API, worker surface, and dependency graph.